Supplementary Information: QEC-R01

Vahif Khalilov

25 October 2026

1 Supplementary Information

This supplement reproduces the exact source of `src/calibration.py` as archived with the manuscript. The simulation uses an X-only, ideal-syndrome, three-qubit code-capacity model. The original Gemini review has not been independently authenticated, and QuTiP validation was not executed.

Source listing: src/calibration.py

```
#!/usr/bin/env python3
"""Phase 2 H1: finite-shot calibration of a 3-qubit repetition-code MAP decoder.

For each of R independent calibration data sets, measure physical bit 0 of a
prepared |000> system after independent flips and paired (0,1) burst. The
single-bit marginal is Bernoulli(q0) where q0=p+(1-2p)*b.
Its estimator b_hat=(sample_mean-p)/(1-2p) is clipped to [0,0.08].
The decoder is evaluated against the *exact eight-mask distribution* on
independent, unobserved evaluation trials, not on training labels.
"""
import argparse
import json
import math
import platform
import random
import sys
from pathlib import Path

from pilot import BURST, decode_table, exact_failure_rate, min_weight_table, syndrome

P = 0.02
TRUE_B = 0.04
SIZES = (0, 50, 200, 1000, 5000)
REPLICATES = 2000
MAX_B = 0.08
SEED = 20261025


def estimate_b_from_ones(ones: int, n: int, p: float=P) -> float:
    if n <= 0 or not 0 <= ones <= n or not 0 <= p < .5:
        raise ValueError('invalid calibration parameters')
    return max(0.0, min(MAX_B, (ones/n-p)/(1-2*p)))


def wilson95(k: int, n: int):
    if n <= 0:
        return [None, None]
    z = 1.959963984540054
    x = k/n
    denom = 1 + z*z/n
    center = (x + z*z/(2*n))/denom
    delta = z*math.sqrt((x*(1-x)+z*z/(4*n))/n)/denom
    return [max(0., center-delta), min(1., center+delta)]


def run(replicates=REPLICATES, seed=SEED):
    if replicates < 1:
        raise ValueError('replicates must be positive')
```

```
    base = min_weight_table()
    oracle = decode_table(P, TRUE_B)
    s_ambiguous = syndrome(BURST)
    baseline_rate = exact_failure_rate(P, TRUE_B, base)
    oracle_rate = exact_failure_rate(P, TRUE_B, oracle)
    q0 = P+(1-2*P)*TRUE_B
    assert abs(q0-0.0584) < 1e-14
    assert oracle[s_ambiguous] == BURST

    data = []
    for n in SIZES:
        rng = random.Random(seed + n)
        selection_count = 0
        rates = []
        estimates = []
        for _ in range(replicates):
            if n == 0:
                current = base
                estimate = None
            else:
                ones = sum(rng.random() < q0 for _ in range(n))
                estimate = estimate_b_from_ones(ones, n)
                current = decode_table(P, estimate)
                estimates.append(estimate)
            selection_count += (current[s_ambiguous] == oracle[s_ambiguous])
            rates.append(exact_failure_rate(P, TRUE_B, current))
        mean_rate = sum(rates)/replicates
        fraction = selection_count/replicates
        # Under the fixed model the MAP table is selected by a Bernoulli branch;
        # exact outcome rates are the two possible conditional values. This CI
        # propagates Wilson bounds for calibration-replicate success frequency.
        wl, wu = wilson95(selection_count, replicates)
        mean_ci = [baseline_rate+(oracle_rate-baseline_rate)*wu,
                   baseline_rate+(oracle_rate-baseline_rate)*wl]
        data.append({
            'N': n, 'replicates': replicates,
            'number_selecting_oracle_ambiguous_branch': selection_count,
            'oracle_branch_fraction': fraction,
            'oracle_branch_wilson95': [wl, wu],
            'average_exact_logical_error_rate': mean_rate,
            'average_exact_logical_error_rate_wilson_derived95': mean_ci,
            'improvement_vs_min_weight_percentage_points': 100*(baseline_rate-mean_rate),
            'mean_estimated_b': (sum(estimates)/len(estimates)) if estimates else None,
        })
    selected = next(row for row in data if row['N'] == 1000)
    return {
        'study': 'QEC-R01 Phase 2 H1 preregistered primary condition',
        'model': '3-qubit X-only repetition code; ideal syndrome readout; independent calibration on known |000> states',
        'p':P,'true_pair_b': TRUE_B,'calibration_first_bit_marginal':q0,
        'estimated_b_clipping': [0,MAX_B],
        'N_values':list(SIZES), 'replicates_per_N':replicates, 'random_seed':seed,
        'python':platform.python_version(),
        'calibration_measurement': 'physical qubit 0, single-shot computational-basis measurement of prepared |000> state',
        'logical_error_evaluation': 'exact 8-mask enumeration on true distribution; no re-use of calibration samples',
        'exact_minimum_weight_LER':baseline_rate,'exact_oracle_MAP_LER':oracle_rate,
        'decision_threshold_b':P*(1-P),
        'results':data,
        'H1_at_N1000': {
            'requirement_branch_fraction_at_least_0_95': selected['oracle_branch_fraction'] >= 0.95,
            'requirement_average_LER_below_baseline': selected['average_exact_logical_error_rate'] < baseline_rate,
            'H1_criteria_met': selected['oracle_branch_fraction'] >= 0.95 and selected['average_exact_logical_error_rate'] < baseline_rate,
            'statistical_note': 'Two pre-registered descriptive criteria on 2000 replicates; this is not an independently externally audited hypothesis test.'
        },
        'limitations': [
            'No circuit-level measurement, leakage, Z/Y errors, drift, or actual hardware',
            'Known p; destructive reference |000> calibration; clip b to [0,0.08]',
            'Monte Carlo randomization is for calibration only; final LER evaluated analytically',
            'Only true b=0.04 here; full preregistered b grid and independent eval Monte Carlo remain NOT RUN',
        ]
    }


def main():
    parser=argparse.ArgumentParser()
    parser.add_argument('--replicates',type=int,default=REPLICATES)
    parser.add_argument('--seed',type=int,default=SEED)
    parser.add_argument('--output',default='results/calibration.json')
    args=parser.parse_args()
    report=run(args.replicates,args.seed)
    out=Path(args.output)
    out.parent.mkdir(parents=True,exist_ok=True)
    out.write_text(json.dumps(report,indent=2,ensure_ascii=False,allow_nan=False)+'\n',encoding='utf-8')
    print(f'QEC-R01 Phase 2 | p={P:.4f} b_true={TRUE_B:.4f} | R={args.replicates} | seed={args.seed}')
```

```
    print(f'exact minimum-weight LER={report["exact_minimum_weight_LER"]:.6f}; oracle MAP LER={report["exact_oracle_MAP_LER"]:.6f}')
    print('N     oracle_branch_frac   avg_exact_LER   gain_pp')
    for row in report['results']:
        print(f'{row["N"]:5d}    {row["oracle_branch_fraction"]:.4f}          {row["average_exact_logical_error_rate"]:.6f}       {row["improvement_vs_min_weight_percentage_points"]:+.4f}')
    print('H1 criteria met:',report['H1_at_N1000']['H1_criteria_met'])
    print('JSON:',out)

if __name__=='__main__':
    main()
```